



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251187
Nama Lengkap	Benedictina Andhika Chinor Jodie Soesila
Minggu ke / Materi	06 / Struktur Kontrol Perulangan

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Definisi Perulangan

Sebuah program dapat dijalankan dengan urutan langkah, percabangan, pengulangan, atau gabungan dari ketiganya. Mekanisme ini disebut struktur kontrol. Pengulangan dipakai ketika program perlu:

1. Menjalankan proses yang sama berulang kali.
2. Melakukan tahapan yang memiliki pola serupa.
3. Mengakses kumpulan data dalam struktur tertentu, misalnya List, Tuple, Queue, Stack, dan berbagai struktur data lainnya.

Dalam Python, pengulangan bisa dilakukan dengan `for`, `while`, maupun rekursi. Pada pembahasan kali ini, fokus diberikan pada penggunaan perulangan `for` dan `while`.

Bentuk Perulangan `for`

Dalam Python, perulangan dapat ditulis menggunakan **`for`** maupun **`while`**. Struktur **`for`** biasanya dipakai ketika jumlah pengulangan sudah ditentukan sejak awal. Contohnya, membaca isi dari 10 file teks: meskipun isi tiap file berbeda, proses pembacaannya tetap sama, dilakukan berurutan dari file pertama hingga file kesepuluh.

Selain itu, **`for`** juga digunakan ketika operasi dilakukan pada suatu rentang data atau nilai. Misalnya, menghitung jumlah 100 bilangan pertama dengan cara menambahkan angka 1, 2, 3, dan seterusnya sampai 100. Dengan kata lain, perulangan berjalan dalam rentang tertentu.

Untuk mempermudah, Python menyediakan fungsi **`range()`** yang menghasilkan deret angka sesuai kebutuhan:

1. **`range(stop)`** → menghasilkan angka dari 0 hingga `stop-1`. Contoh: **`range(6)`** menghasilkan 0 sampai 5.
2. **`range(start,stop,step)`** → menghasilkan angka dari `start` hingga `stop` dengan kenaikan sesuai nilai `step`.

Contoh berikut menunjukkan bagaimana menampilkan bilangan dari 1 hingga 100 dengan memanfaatkan perulangan **`for`** bersama fungsi **`range()`** di Python:

```
for i in range(1, 101):  
    print(i)
```

Pada program tersebut digunakan perulangan **for** dengan fungsi **range()**, dimulai dari angka 1 (sebagai *start*) hingga 101 (*stop-1*), dengan langkah default sebesar 1. Variabel **i** berperan sebagai *counter*, di mana nilainya akan bertambah secara berurutan sesuai deret angka yang dihasilkan oleh fungsi **range()**. Namun, dalam kondisi tertentu ketika variabel *counter* tidak diperlukan, bentuk perulangan bisa ditulis lebih sederhana. Misalnya, cukup menggunakan struktur **for** tanpa memanfaatkan nilai variabel di dalam blok perulangan.

```
for _ in range(1, 101):  
    print('Hello World')
```

Program tersebut mencetak "**Hello World**" sebanyak 100 kali tanpa menggunakan nilai *counter*, karena hanya mengulang perintah yang sama berulang kali.

STOP NEGATIF

Perhatikan contoh perulangan **for** berikut ini yang akan menampilkan seluruh deretan bilangan genap mulai dari angka 2 sampai dengan 100:

```
for i in range(2, 101, 2):  
    print(i)
```

Perulangan dilakukan pada rentang 2-100 , dengan pertambahan 2 tiap indeksinya, sehingga outpunya 2,4, 6, 8, 10, 12, ..., 100.

```
for i in range(100, 1, -2):  
    print(i)
```

Program tersebut akan menampilkan bilangan genap mulai dari 100, 98, 96, 94, ..., sampai 2.

Bentuk Perulangan While

Bentuk **while** umumnya dipakai ketika jumlah pengulangan belum ditentukan sebelumnya.

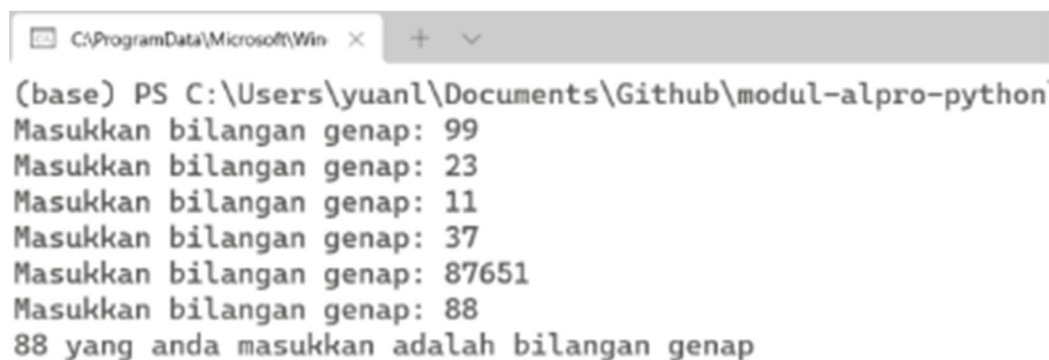
Secara umum, struktur perulangan **while** dituliskan dengan pola seperti berikut:

```
<start>  
while <stop>:  
    operation  
    operation  
    <step (optional)>
```

Contoh berikut menunjukkan perulangan dengan menggunakan while, yang berfungsi untuk memastikan bahwa input yang diberikan oleh pengguna merupakan bilangan genap:

```
bilangan = 0
genap = False
while genap == False:
    bilangan = int(input('Masukkan bilangan genap: '))
    if bilangan % 2 == 0:
        genap = True
print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Output dari program tersebut ditunjukkan pada Gambar berikut ini. Awalnya pengguna memasukkan bilangan ganjil, sehingga program terus meminta input bilangan genap. Proses berhenti ketika pengguna memasukkan angka 88, yang merupakan bilangan genap. Situasi seperti ini sangat tepat menggunakan perulangan **while**, karena jumlah percobaan memasukkan bilangan ganjil tidak dapat diprediksi sebelumnya.



```
(base) PS C:\Users\yuanl\Documents\Github\modul-alpro-python
Masukkan bilangan genap: 99
Masukkan bilangan genap: 23
Masukkan bilangan genap: 11
Masukkan bilangan genap: 37
Masukkan bilangan genap: 87651
Masukkan bilangan genap: 88
88 yang anda masukkan adalah bilangan genap
```

Gambar 1.1 Penggunaan while untuk mengambil input dari pengguna sampai sesuai dengan permintaan. Sumber: <https://sl1nk.com/pLcFf>

Penggunaan Break dan Continue

Perulangan dapat dikendalikan menggunakan **break** maupun **continue**. Secara umum, **break** dipakai untuk menghentikan jalannya perulangan, sedangkan **continue** digunakan agar perulangan langsung berlanjut ke iterasi berikutnya. Perhatikan contoh program berikut yang menampilkan bilangan dari 1 sampai 10:

```
for i in range(1, 11):
    print(i)
print('Selesai')
```

Program tersebut akan menampilkan deretan bilangan dari 1 sampai 10, lalu pada baris terakhir muncul tulisan 'Selesai'. Jika diinginkan agar perulangan yang seharusnya berjalan hingga angka 10 dihentikan lebih awal, misalnya saat mencapai angka 5, maka diperlukan penggunaan perintah **break** dengan kondisi tertentu seperti pada program berikut ini:

```
for i in range(1, 11):  
    if i == 5:  
        break  
    else:  
        print(i)  
print('Selesai')
```

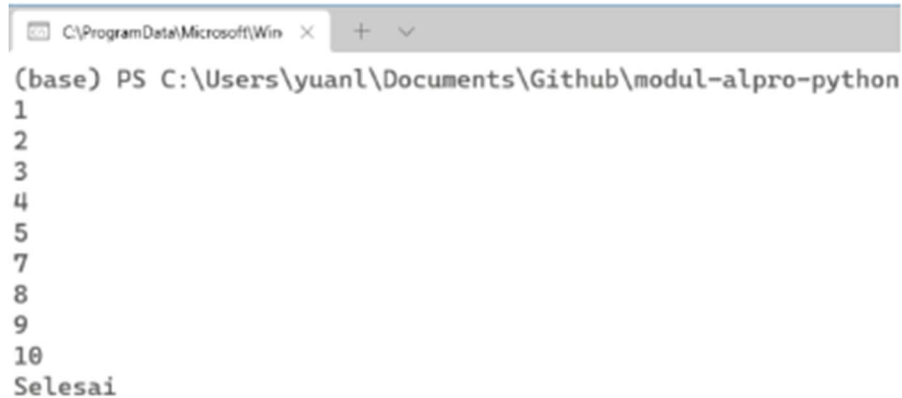
Ketika nilai **i = 5**, ekspresi logika **i == 5** akan bernilai **True**, sehingga program segera menjalankan instruksi **break**. Akibatnya, perulangan langsung dihentikan dan eksekusi program berlanjut ke baris setelah perulangan, yaitu menampilkan tulisan "**Selesai**".

Perbedaan utama antara **break** dan **continue** adalah: **break** digunakan untuk menghentikan perulangan sepenuhnya, sedangkan **continue** dipakai ketika ingin melewati satu tahap perulangan saat ini dan langsung melanjutkan ke iterasi berikutnya.

Sebagai contoh, berikut adalah program yang menampilkan angka dari 1 sampai 10, tetapi dengan syarat angka 6 tidak ditampilkan karena dilewati menggunakan **continue**.

```
for i in range(1, 11):  
    if i == 5:  
        break  
    else:  
        print(i)  
print('Selesai')
```

Output yang dihasilkan yaitu menampilkan angka dari 1 sampai 10, tetapi melewatkan angka 6.



```
(base) PS C:\Users\yuanl\Documents\Github\modul-alpro-python
1
2
3
4
5
7
8
9
10
Selesai
```

Gambar 1.2 Penggunaan continue untuk melewati iterasi saat counter(i) bernilai 6. *Sumber:*

<https://sl1nk.com/pLcFf>

Konversi dari Bentuk for Menjadi Bentuk while

Sebagian besar bentuk perulangan **for** sebenarnya dapat dikonversi menjadi perulangan **while**.

Pada dasarnya, baik **for** maupun **while** memiliki komponen penting yang sama, yaitu:

1. Harus ada nilai awal yang digunakan untuk memulai jalannya perulangan.
2. Harus ada nilai akhir yang menjadi batas untuk menghentikan perulangan.
3. Harus ada langkah atau kenaikan nilai, agar iterasi dari nilai awal dapat terus berjalan hingga mencapai nilai akhir.

Sebagai ilustrasi, misalkan terdapat sebuah perulangan **for** sederhana seperti berikut:

```
for i in range(1, 11):
    print(i)
```

Dari perulangan tersebut dapat diidentifikasi bahwa proses dimulai dari angka 1, berakhir pada angka 10, dengan langkah atau *step* sebesar 1. Konversi ke bentuk **while** dapat dilakukan dengan mudah karena prinsip kerjanya sama, yaitu memulai dari nilai awal, menambah sesuai langkah, dan berhenti ketika mencapai nilai akhir. Dengan cara ini, perulangan **while** akan menghasilkan keluaran yang identik dengan versi **for**, yakni menampilkan bilangan dari 1 hingga 10 secara berurutan. Perbedaan utama hanya terletak pada cara penulisan sintaks, di mana **for** lebih ringkas karena langsung menyertakan nilai awal, akhir, dan langkah dalam satu baris, sedangkan **while** membutuhkan inisialisasi variabel, kondisi logika, serta pernyataan untuk menambah nilai

agar perulangan tetap berjalan. Walaupun berbeda bentuk, keduanya tetap memberikan hasil yang sama sesuai kebutuhan program.

```
i = 1 //awal
while i <= 10: //kondisi akhir
    print(i)
    i = i + 1 //step
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.\

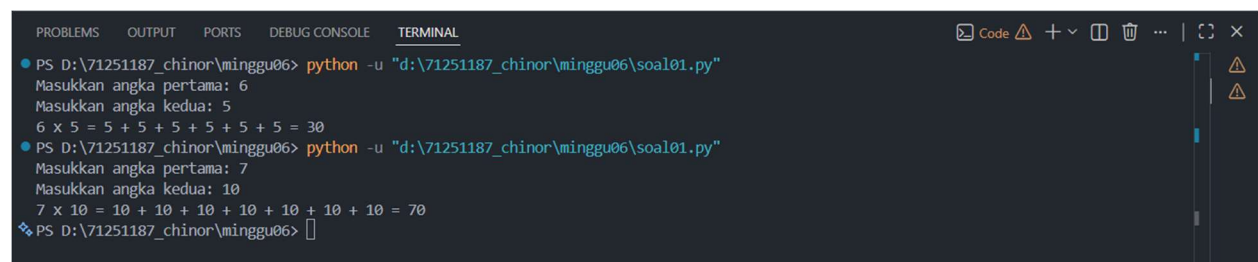
Link Github: https://github.com/chinorjodie/71251187_chinor.git

SOAL 1

Source code:

```
def perkalian_cuy(a, b):  
    hasil = 0  
    ekspresi = []  
    for i in range(a):  
        hasil += b  
        ekspresi.append(str(b))  
    penjumlahan_str = " + ".join(ekspresi)  
    print(f"{a} x {b} = {penjumlahan_str} = {hasil}")  
  
a = int(input("Masukkan angka pertama: "))  
b = int(input("Masukkan angka kedua: "))  
  
perkalian_cuy(a,b)
```

Output:



```
PROBLEMS OUTPUT PORTS DEBUG CONSOLE TERMINAL  
PS D:\71251187_chinor\minggu06> python -u "d:\71251187_chinor\minggu06\soal01.py"  
Masukkan angka pertama: 6  
Masukkan angka kedua: 5  
6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30  
PS D:\71251187_chinor\minggu06> python -u "d:\71251187_chinor\minggu06\soal01.py"  
Masukkan angka pertama: 7  
Masukkan angka kedua: 10  
7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70  
PS D:\71251187_chinor\minggu06>
```

Penjelasan:

Program ini mendefinisikan fungsi **perkalian_cuy(a, b)** yang bertujuan menampilkan hasil perkalian dalam bentuk penjumlahan berulang. Di dalam fungsi, variabel **hasil** diinisialisasi dengan 0, lalu dilakukan perulangan sebanyak nilai **a**. Setiap iterasi menambahkan nilai **b** ke **hasil** dan menyimpan **b** ke dalam list **ekspresi**. Setelah perulangan selesai, list tersebut digabungkan

menjadi string dengan tanda " + " sehingga terlihat jelas bahwa perkalian sebenarnya adalah penjumlahan berulang. Program kemudian mencetak hasil dalam format: **a x b = ekspresi penjumlahan = hasil**.

Bagian akhir meminta pengguna memasukkan dua angka melalui **input**, lalu memanggil fungsi untuk menampilkan hasil perkalian beserta bentuk penjumlahan yang merepresentasikannya. Dengan cara ini, pengguna tidak hanya melihat hasil akhir, tetapi juga proses perkalian yang ditunjukkan.

SOAL 2

Source code:

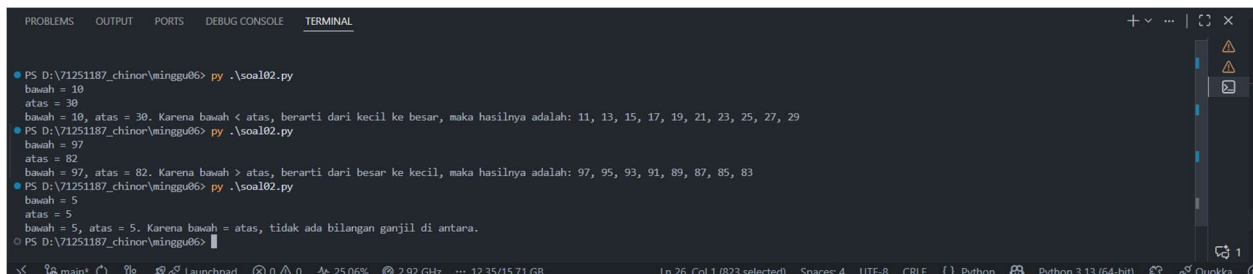
```
def ganjil_cuy(bawah,atas):
    hasil = []
    if bawah < atas:
        ket_1 = "dari kecil ke besar"
        ket_2 = "bawah < atas"
        for i in range(bawah, atas + 1):
            if i % 2 == 1:
                hasil.append(str(i))
    elif bawah > atas:
        ket_1 = "dari besar ke kecil"
        ket_2 = "bawah > atas"
        for i in range(bawah, atas - 1, -1):
            if i % 2 == 1 :
                hasil.append(str(i))
    else:
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah = atas, tidak ada bilangan ganjil di antara.")
        return

    if hasil:
        print(f"bawah = {bawah}, atas = {atas}. Karena {ket_2}, berarti {ket_1}, maka hasilnya adalah: {' '.join(hasil)}")

bawah = int(input("bawah = "))
```

```
atas = int(input("atas = "))  
  
ganjil_cuy(bawah,atas)
```

Output:



```
PS D:\71251187_chinor\minggu06> py .\soal02.py  
bawah = 10  
atas = 30  
bawah = 10, atas = 30. Karena bawah < atas, berarti dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29  
PS D:\71251187_chinor\minggu06> py .\soal02.py  
bawah = 97  
atas = 82  
bawah = 97, atas = 82. Karena bawah > atas, berarti dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83  
PS D:\71251187_chinor\minggu06> py .\soal02.py  
bawah = 5  
atas = 5  
bawah = 5, atas = 5. Karena bawah = atas, tidak ada bilangan ganjil di antara.  
PS D:\71251187_chinor\minggu06>
```

Penjelasan:

Program di atas mendefinisikan sebuah fungsi bernama **ganjil_cuy(bawah, atas)** yang digunakan untuk menampilkan bilangan ganjil dalam rentang tertentu sesuai dengan nilai batas bawah dan batas atas yang dimasukkan pengguna. Pertama, fungsi membuat list kosong bernama **hasil** untuk menampung bilangan ganjil. Jika nilai **bawah** lebih kecil daripada **atas**, maka perulangan dilakukan dari kecil ke besar, dan setiap angka ganjil dalam rentang tersebut dimasukkan ke dalam list. Sebaliknya, jika nilai **bawah** lebih besar daripada **atas**, maka perulangan berjalan dari besar ke kecil, dan angka ganjil tetap dikumpulkan. Namun, jika nilai **bawah** sama dengan **atas**, program langsung mencetak pesan bahwa tidak ada bilangan ganjil di antara keduanya. Setelah perulangan selesai, jika ada bilangan ganjil yang ditemukan, program akan mencetak hasil lengkap dengan keterangan arah perulangan. Pada bagian akhir, pengguna diminta memasukkan nilai **bawah** dan **atas** melalui input, lalu fungsi dijalankan untuk menampilkan bilangan ganjil sesuai dengan kondisi yang diberikan.

SOAL 3

Source code:

```
def ips_lu():  
    jumlah_mk = int(input("Berapa jumlah mata kuliah? "))  
    total_sks = jumlah_mk * 3  
    total_nilai = 0
```

```

for i in range (1, jumlah_mk + 1):
    nilai = input(f"Nilai MK {i}: ")

    if nilai == "A":
        bobot = 4
    elif nilai == "B":
        bobot = 3
    elif nilai == "C":
        bobot = 2
    elif nilai == "D":
        bobot = 1
    else:
        pass
    total_nilai += bobot * 3

ips = total_nilai/total_sks
print(f"IPS Anda semester ini {ips:.2f}")

ips_lu()

```

Output:

```

D:\1521181~chitua\wtu88nge> python3 1521181~chitua\wtu88nge\ips_lu.py
jumlah_mk: 5
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: C
Nilai MK 4: D
Nilai MK 5: A
IPS semester ini: 3.20
D:\1521181~chitua\wtu88nge>

```

Penjelasan:

Program di atas mendefinisikan sebuah fungsi bernama **ips_lu()** yang digunakan untuk menghitung Indeks Prestasi Semester (IPS) berdasarkan jumlah mata kuliah dan nilai yang diperoleh. Pertama, pengguna diminta memasukkan jumlah mata kuliah, lalu program menghitung total SKS dengan mengalikan jumlah mata kuliah tersebut dengan 3 (diasumsikan setiap mata kuliah memiliki 3 SKS). Selanjutnya, program melakukan perulangan untuk setiap mata kuliah dan meminta input nilai berupa huruf (A, B, C, atau D). Nilai huruf tersebut kemudian dikonversi menjadi bobot angka: A = 4, B = 3, C = 2, dan D = 1. Bobot

tersebut dikalikan dengan 3 (jumlah SKS per mata kuliah) dan ditambahkan ke total nilai. Setelah semua nilai diproses, IPS dihitung dengan membagi total nilai dengan total SKS. Terakhir, program menampilkan hasil IPS dengan format dua angka di belakang koma. Dengan demikian, fungsi ini memberikan gambaran sederhana bagaimana IPS dihitung dari nilai huruf yang dimasukkan pengguna.